Github 账号：sparkle37-misc

密码学实验博客的链接：**https://github.com/sparkle37-misc/Cryptography**

实验题目：

1. Many Time Pad

2. 编写一个程序，使能够"破解"使用类似维基解密的密码生成的密码文本，在这种密码中使用字节 XOR 而不是 26 模加法。

3. The Cryptopals Crypto Challenges

4. 破解 SHA1 加密

实验摘要：

1.学习Many time pad 的攻击方法；

2.Crack Vigenere-like cipher ： 学习python 下的密码函数基本用法，如 base64 编码，hex 转换，字节与字符的转换；

3.学习词频分析，学习攻击维吉尼亚密码；

4.学习暴力破解的方法与思路。

题目描述

1.Crack Many Time Pad：

一次一密是完全安全的，但如果重复使用密钥会存在安全问题，尝试攻破多次使用一个密钥加密的加密方案。

2.Crack Vigenere-like cipher：

"破解 "使用类维基解密的密码，在这种密码中使用字节 XOR 而不非 26 模加法。

3.Cryptopal/Set1:

Convert hex to base64；
Fixed XOR；
Single-byte XOR cipher；
Detect single-character XOR；
Implement repeating-key XOR；
Break repeating-key XOR。

相关密码学基础编程用法与函数，以及破解重复加密。

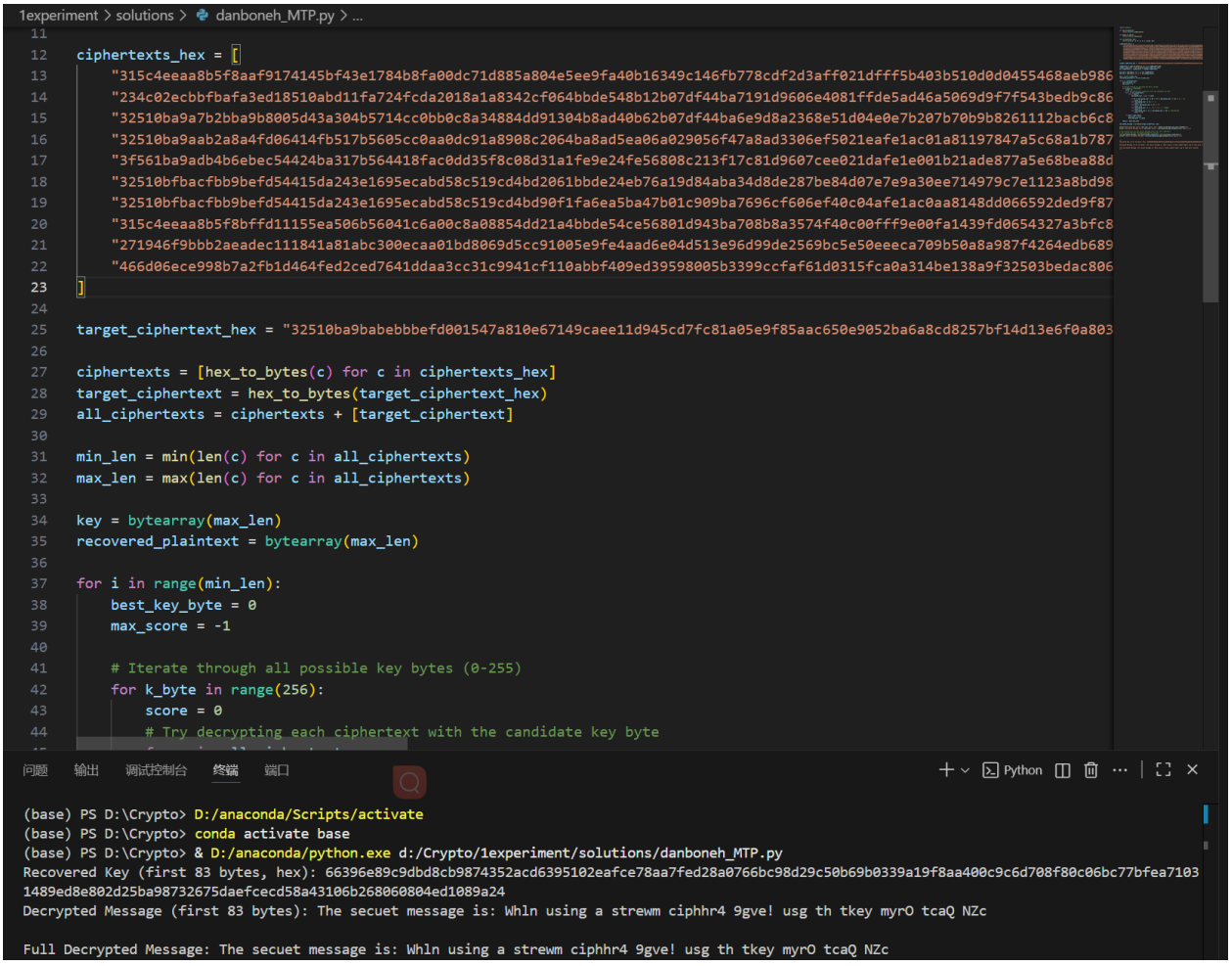
4.Cracking SHA1-Hashed Passwords

知道通过键盘输入的密码的hash，以及可能的输入字符，请破解这个密码。

过程

1.

空格字符的ASCII码为二进制0010 0000。在英文字母的编码中，大写字母A至Z分布于0100 0001至0101 1010，而小写字母a至z则位于0110 0001至0111 1010区间。观察这一编码规律可以发现：任一字母与空格进行异或运算，会实现其大小写形态的切换；但若两个字母直接异或，所得结果通常不再属于字母编码范围。基于上述特性，在密码分析中可采取以下方法推测密钥：首先对两段密文执行异或操作，由于该结果等同于对应明文异或的结果，可通过统计分析识别出明文中的空格位置。一旦确定空格所在，即可用该处密文与空格编码（0010 0000）进行异或，从而直接计算出密钥在该字节的值。



```
1experiment > solutions > danboneh_MTP.py > ...
11
12  ciphertexts_hex = [
13      "315c4eeaa8b5f8aaf9174145bf43e1784b8fa00dc71d885a804e5ee9fa40b16349c146fb778cdf2d3aff021dfff5b403b510d0d0455468aeb986
14      "234c02ecbbfbafa3ed18510abd11fa724fcda2018a1a8342cf064bbde548b12b07df44ba7191d9606e4081ffde5ad46a5069d9f7f543bedb9c86
15      "32510ba9a7b2bba9b8005d43a304b5714cc0bb0c8a34884dd91304b8ad40b62b07df44ba6e9d8a2368e51d04e0e7b207b70b9b8261112bacb6c8
16      "32510ba9aab2a8a4fd06414fb517b5605cc0aa0dc91a8908c2064ba8ad5ea06a029056f47a8ad3306ef5021eafe1ac01a81197847a5c68a1b787
17      "3f561ba9adb4b6ebec54424ba317b564418fac0dd35f8c08d31a1fe9e24fe56808c213f17c81d9607cee021dafe1e001b21ade877a5e68bea88d
18      "32510bfbacfbb9befd54415da243e1695ecabd58c519cd4bd2061bbde24eb76a19d84aba34d8de287be84d07e7e9a30ee714979c7e1123a8bd98
19      "32510bfbacfbb9befd54415da243e1695ecabd58c519cd4bd90f1fa6ea5ba47b01c909ba7696cf606ef40c04afe1ac0aa8148dd066592ded9f87
20      "315c4eeaa8b5f8bffd11155ea506b56041c6a00c8a08854dd21a4bbde54ce56801d943ba708b8a3574f40c00fff9e00fa1439fd0654327a3bfc8
21      "271946f9bbb2aeadec111841a81abc300ecaa01bd8069d5cc91005e9fe4aad6e04d513e96d99de2569bc5e50eeeca709b50a8a987f4264edb689
22      "466d06ece998b7a2fb1d464fed2ced7641ddaa3cc31c9941cf110abbf409ed39598005b3399ccfaf61d0315fca0a314be138a9f32503bedac806
23  ]
24
25  target_ciphertext_hex = "32510ba9babebbbefd001547a810e67149caee11d945cd7fc81a05e9f85aac650e9052ba6a8cd8257bf14d13e6f0a803
26
27  ciphertexts = [hex_to_bytes(c) for c in ciphertexts_hex]
28  target_ciphertext = hex_to_bytes(target_ciphertext_hex)
29  all_ciphertexts = ciphertexts + [target_ciphertext]
30
31  min_len = min(len(c) for c in all_ciphertexts)
32  max_len = max(len(c) for c in all_ciphertexts)
33
34  key = bytearray(max_len)
35  recovered_plaintext = bytearray(max_len)
36
37  for i in range(min_len):
38      best_key_byte = 0
39      max_score = -1
40
41      # Iterate through all possible key bytes (0-255)
42      for k_byte in range(256):
43          score = 0
44          # Try decrypting each ciphertext with the candidate key byte

问题  输出  调试控制台  终端  端口                                    Python

(base) PS D:\Crypto> D:/anaconda/Scripts/activate
(base) PS D:\Crypto> conda activate base
(base) PS D:\Crypto> & D:/anaconda/python.exe d:/Crypto/1experiment/solutions/danboneh_MTP.py
Recovered Key (first 83 bytes, hex): 66396e89c9dbd8cb9874352acd6395102eafce78aa7fed28a0766bc98d29c50b69b0339a19f8aa400c9c6d708f80c06bc77bfea7103
1489ed8e802d25ba98732675daefcecd58a43106b268060804ed1089a24
Decrypted Message (first 83 bytes): The secuet message is: Whln using a strewm ciphhr4 9gve! usg th tkey myrO tcaQ NZc

Full Decrypted Message: The secuet message is: Whln using a strewm ciphhr4 9gve! usg th tkey myrO tcaQ NZc
```

对密钥进行适当调整，得到正确结果

```
key='66396e89c9dbd8cc9874352acd6395102eafce78aa7fed08a07f6bc98d29c50b69b0339a19f8aa401a9c6d708f80c066c763fef0123148cdd8e80
ciphertext='32510ba9babebbbefd001547a810e67149caee11d945cd7fc81a05e9f85aac650e9052ba6a8cd8257bf14d13e6f0a803b54fde9e77472d
plaintext = xor(bytes.fromhex(key),bytes.fromhex(ciphertext))

print(plaintext)
```

问题 16 输出 调试控制台 终端 端口

```
b'The secret message is: when using a stream cipher, never use the key more than once'
```

2.

①生成字符集合，判断密文被解密后生成的字符是否为合理的明文字符

```
def generate_required_chars():
    required_chars = [chr(32), chr(33), chr(34), chr(44), chr(45), chr(46), chr(58), chr(63), chr(95)]
    required_chars += [chr(x) for x in range(65, 91)]
    required_chars += [chr(x) for x in range(97, 123)]
    return required_chars
```

②通过遍历所有可能的密钥并验证字符合法性，找到符合要求的候选密钥

```
def find_keys(sub_arr, required_chars):
    all_keys = list(range(0x00, 0xff + 1))
    res_keys = all_keys.copy()

    for i in all_keys:
        for s in sub_arr:
            if chr(s ^ i) not in required_chars:
                res_keys.remove(i)
                break
    return res_keys
```

③遍历不同的密钥长度

```
def analyze_key_lengths(arr, max_key_length=13):
    required_chars = generate_required_chars()
    for key_length in range(1, max_key_length + 1):
        for class_number in range(key_length):
            sub_arr = arr[class_number::key_length]
            res_keys = find_keys(sub_arr, required_chars)
            print(f'key_length= {key_length}, class_number= {class_number}, keys= {res_keys}')

            if len(res_keys) > 1:
                for x in res_keys:
                    print(f'Key: {x}')
                    for s in sub_arr:
                        print(chr(s ^ x), end='')
                    print("\n")
```

④使用找到的密钥解密密文，输出最终的明文

```
key_length= 7, class_number= 0, keys= [186]
key_length= 7, class_number= 1, keys= [29, 31]
key_length= 7, class_number= 2, keys= [145]
key_length= 7, class_number= 3, keys= [178]
key_length= 7, class_number= 4, keys= [83]
key_length= 7, class_number= 5, keys= [205]
key_length= 7, class_number= 6, keys= [62]
```

```
def decrypt_text(arr, keys):
    plaintext = ""
    for i in range(len(arr)):
        plaintext += chr(arr[i] ^ keys[i % len(keys)])
    return plaintext
```

```
keys1 = [186, 29, 145, 178, 83, 205, 62]
plaintext1 = decrypt_text(arr, keys1)
print("Decrypted Text1:", plaintext1)

keys2 = [186, 31, 145, 178, 83, 205, 62]
plaintext2 = decrypt_text(arr, keys2)
print("Decrypted Text2", plaintext2)

问题 16  输出  调试控制台  终端  端口

key_length= 13, class_number= 12, keys= []
Decrypted Text1: Cpyptogrcphy is"the prcctice cnd stufy of tgchniqugs for,"among mther tjings, qecure aommuniaation kn the rresencg of atvackers, Cryptmgraphy"has beg
n used"for huldreds,"if not"thousalds, of"years,"but trcditioncl crypvosysteos were"designgd and gvaluatgd in a"fairly"ad hoc"manner, For ezample,"the Vieenere gncryp
tkon schgme was"thoughv to be"secure"for deaades adter it"was intented,"but we"now knmw, and"this ezercise"demonsvrates,"that iv can bg brokel very gasily.
Decrypted Text2 Cryptography is the practice and study of techniques for, among other things, secure communication in the presence of attackers. Cryptography has been
 used for hundreds, if not thousands, of years, but traditional cryptosystems were designed and evaluated in a fairly ad hoc manner. For example, the Vigenere encrypt
ion scheme was thought to be secure for decades after it was invented, but we now know, and this exercise demonstrates, that it can be broken very easily.
```

分析得到密钥为[186, 31, 145, 178, 83, 205, 62]

明文为 Cryptography is the practice and study of techniques for, among other things, secure communication in the presence of attackers. Cryptography has been used for hundreds, if not thousands, of years, but traditional cryptosystems were designed and evaluated in a fairly ad hoc manner. For example, the Vigenere encryption scheme was thought to be secure for decades after it was invented, but we now know, and this exercise demonstrates, that it can be broken very easily.

3.

①计算密文块之间的汉明距离，找出最有可能的密钥长度。将密文按照不同的候选密钥长度进行分块，将每个块的前 **keysize** 个字节两两配对，计算其平均汉明

距离，并将这个距离归一化。选择平均距离最小的密钥长度作为最终的密钥长度，因为正确的密钥长度会使加密文本块相似，导致较小的汉明距离。

```
def guess_key_length(ciphertext, min_keysize=2, max_keysize=40):
    keysize_scores = []
    for keysize in range(min_keysize, max_keysize + 1):
        chunks = [ciphertext[i:i + keysize] for i in range(0, len(ciphertext), keysize)]
        distances = []
        for i in range(len(chunks) - 1):
            distance = hamming_distance(chunks[i], chunks[i + 1])
            distances.append(distance / keysize)
        avg_distance = sum(distances) / len(distances)
        keysize_scores.append((keysize, avg_distance))
    return min(keysize_scores, key=lambda x: x[1])[0]
```

②在确定了可能的加密密钥长度后，下一步是将原始密文按此长度进行分块和转置。这样，每个新的密文块都将仅由密钥中的一个特定字节进行加密。随后，对每个转置后的密文块执行独立的穷举式单字节异或（XOR）分析。具体操作为：遍历 0 到 255 之间的所有候选字节值**，将密文块与之异或解密。通过字母频率分析对产生的候选明文进行拟合度评估——通常是计算其字符分布与标准英语文本（例如，常用字母如 'e' 和空格的频率）的相似程度。最终，得分最高（即与目标语言特征最匹配）的那个单字节值，即被认定为该密文块对应的正确密钥字节。

```
def break_repeating_key_xor(ciphertext):
    keysize = guess_key_length(ciphertext)
    key = b''

    for i in range(keysize):
        block = bytes([ciphertext[j] for j in range(i, len(ciphertext), keysize)])
        key += bytes([break_single_byte_xor(block)['key']])

    decrypted_message = repeating_key_xor_decrypt(ciphertext, key)
    return decrypted_message, key
```

③使用完整的密钥对密文进行重复密钥 XOR 解密，输出最终的明文和密钥。

```
def repeating_key_xor_decrypt(ciphertext, key):
    return bytes([ciphertext[i] ^ key[i % len(key)] for i in range(len(ciphertext))])
```

```
Decrypted Message: I'm back and I'm ringin' the bell
A rockin' on the mike while the fly girls yell
In ecstasy in the back of me
Well that's my DJ Deshay cuttin' all them Z's
Hittin' hard and the girlies goin' crazy
Vanilla's on the mike, man I'm not lazy.

I'm lettin' my drug kick in
It controls my mouth and I begin
To just let it flow, let my concepts go
My posse's to the side yellin', Go Vanilla Go!

Smooth 'cause that's the way I will be
And if you don't give a damn, then
Why you starin' at me
So get off 'cause I control the stage
There's no dissin' allowed
I'm in my own phase
The girlies sa y they love me and that is ok
And I can dance better than any kid n' play

Stage 2 -- Yea the one ya' wanna listen to
It's off my head so let the beat play through
So I can funk it up and make it sound good
1-2-3 Yo -- Knock on some wood
For good luck, I like my rhymes atrocious
Supercalafragilisticexpialidocious
I'm an effect and that you can bet
I can take a fly girl and make her wet.

I'm like Samson -- Samson to Delilah
There's no denyin', You can try to hang
But you'll keep tryin' to get my style
Over and over, practice makes perfect
But not if you're a loafer.

You'll get nowhere, no place, no time, no girls
Soon -- Oh my God, homebody, you probably eat
Spaghetti with a spoon! Come on and say it!

VIP. Vanilla Ice yep, yep, I'm comin' hard like a rhino
Intoxicating so you stagger like a wino
So punks stop trying and girl stop cryin'
Vanilla Ice is sellin' and you people are buyin'
'Cause why the freaks are jockin' like Crazy Glue
Movin' and groovin' trying to sing along
All through the ghetto groovin' this here song
Now you're amazed by the VIP posse.
```

```
Steppin' so hard like a German Nazi
Startled by the bases hittin' ground
There's no trippin' on mine, I'm just gettin' down
Sparkamatic, I'm hangin' tight like a fanatic
You trapped me once and I thought that
You might have it
So step down and lend me your ear
'89 in my time! You, '90 is my year.

You're weakenin' fast, YO! and I can tell it
Your body's gettin' hot, so, so I can smell it
So don't be mad and don't be sad
'Cause the lyrics belong to ICE, You can call me Dad
You're pitchin' a fit, so step back and endure
Let the witch doctor, Ice, do the dance to cure
So come up close and don't be square
You wanna battle me -- Anytime, anywhere

You thought that I was weak, Boy, you're dead wrong
So come on, everybody and sing this song

Say -- Play that funky music Say, go white boy, go white boy go
play that funky music Go white boy, go white boy, go
Lay down and boogie and play that funky music till you die.

Play that funky music Come on, Come on, let me hear
Play that funky music white boy you say it, say it
Play that funky music A little louder now
Play that funky music, white boy Come on, Come on, Come on
Play that funky music

Key: Terminator X: Bring the noise
```

4.

①指定目标哈希值和字符集合，其中每组字符代表密码中的一个位置，可以有多种可能，基于字符集合，生成所有可能的字符组合和排列，确保涵盖每一种字符排列方式

```python
target_hash = "67ae1a64661ac8b4494666f58c4822408dd0a3e4"

char_sets = [['Q', 'q'], ['W', 'w'], ['%', '5'], ['8', '('], ['=', '0'], ['I', 'i'], ['*', '+'], ['n', 'N']]

def generate_passwords(char_sets):

    for combination in itertools.product(*char_sets):

        for permutation in itertools.permutations(combination):
            yield ''.join(permutation)
```

②对于每个生成的密码，计算其 **SHA-1** 哈希值，并将其与目标哈希值对比。若匹配，则找到密码；否则继续

```
def sha1_encrypt(input_str):
    hash_obj = hashlib.sha1()
    hash_obj.update(input_str.encode('utf-8'))
    return hash_obj.hexdigest()

def find_password(target_hash, char_sets):
    start_time = datetime.datetime.now()

    for password in generate_passwords(char_sets):
        encrypted_password = sha1_encrypt(password)
        if encrypted_password == target_hash:
            end_time = datetime.datetime.now()
            print(f"Password found: {password}")
            print(f"Time taken: {end_time - start_time}")
            return

    print("Password not found.")
```

Password found: (Q=win*5

Time taken: 0:00:08.309465

总结

在本次实验过程中，我学习了如何借助概率分布特征判断文本是否符合英语的语言规律，具体来说，通过比对解密后文本中字母的分布与英语标准字母频率分布的吻合程度，能够快速评估解密结果是否为合理的英文文本。这种基于概率统计的评估方式，显著提升了密钥推测过程的科学性，有效降低了无效尝试的次数。总体而言，上述知识的学习与实际应用，不仅加深了我对密码学中文本分析方法的理解，也大幅提升了解密过程的效率与准确性。

关于第二题**Crack Vigenere-like cipher**，主要通过优秀博主的博客来理解破解的思路，收获匪浅。

参考文献

1.https://blog.soreatu.com/posts/case-study-cryptanalysis-of-vigenere-ciper/

2.https://en.wikipedia.org/wiki/Letter_frequency